The Constructive Transformer

Building a Language Model by Hand — No Gradient Descent, Byte-Exact Arithmetic

TruthSpace Geometric LCM Project

May 2026

Contents

Abstract

We construct a transformer language model entirely by hand — every weight is placed explicitly, not learned. The model operates on a 4-state per-axis alphabet with exact integer arithmetic, achieving byte-exact accuracy on concept arithmetic and subject-verb agreement without any training procedure.

Five core experiments (T4-NEG0, T4-MLP, T4-MLP-CROSS, T4-OLMo2, T4-SVA) progressively build and verify the architecture, and a sixth integration experiment (the T5 series) demonstrates that the construction scales from 24 words to over 1,400 words and from 6 hand-chosen axes to 13 LLM-discovered ones without any change to the underlying substrate. The combined results establish that:

- A transformer's core behaviours — state composition, content routing, cross-axis interaction, and grammatical generation — are structurally inevitable given the architecture, not emergent statistical artefacts.
- The model's accuracy is not approximate but *exact* integer arithmetic, with every logit margin being an integer (typically $+2$ or $+4$).
- The substrate supports the same holographic-gate phenomenology observed in production LLMs (OLMo2-1B, Qwen2-7B), including φ-boundary gating, dark-fringe energy, and sign-over-magnitude advantage.
- The construction scales: at $1{,}438$ words on 13 axes the substrate covers $4^{13} \approx 67$ million distinguishable concept cells while still passing 89% of its multi-step analogy battery and 84% of its self-decode test, with the new axes proposed by an external LLM rather than a human (§8).

All code, weights, and experimental results are included. No libraries beyond NumPy and PyTorch are required.

Chapter 1: Why Build a Transformer by Hand?

A constructive answer to the black-box problem.

1.1 The Black-Box Problem

Modern language models are trained at enormous cost — billions of tokens, thousands of GPU-hours, opaque loss landscapes. The trained model is a black box whose internal representations can only be studied post-hoc, with interpretability tools that are themselves approximate. We see the weights and the activations, but the *grammar* of the computation — the rules by which one state turns into another — has to be reverse-engineered every time.

This paper asks a different question:

> **Can we build a transformer by hand that reproduces the same behaviours?**

A positive answer would establish that those behaviours are *structurally inevitable* given the architecture, not contingent on the specifics of training data, optimiser, or random seed. It would also provide a grounded, fully interpretable substrate for further research: if we know exactly how the model works, we can reason about it formally rather than statistically.

1.2 The Claim

We demonstrate that a transformer LLM is, at minimum, a finite-state machine over a small number of orthogonal axis blocks. Attention routes content across positions; embeddings commit content at positions; MLPs allow content-conditional cross-block routing that pure attention cannot express. The "intelligence" of an LLM, insofar as it lies in concept arithmetic and

grammatical agreement, is *structurally inevitable* given this substrate. It can be constructed by hand, byte-exactly, without any training procedure.

1.3 What Will Be Proved

Across seven chapters, the following propositions are established constructively, by exhibiting explicit weights and exhibiting exact pass-rates:

Proposition	Established in
A 2-bit-per-axis alphabet supports four geometrically distinguishable states with integer margins	§2
The substrate survives and is *enhanced by* a real SwiGLU MLP, reproducing the φ-boundary phenomenology of production LLMs	§3
Attention alone cannot express cross-axis routing; a 6-channel SwiGLU MLP can	§4
The full attention + MLP + embedding stack performs subject-verb agreement, byte-exactly, with hand-placed weights	§5
The construction's predictions are reproduced on a real production LLM (OLMo2-1B)	§6
Every logit and every margin is an *exact integer* under hard attention	§7
The construction scales by composition: a 1,400-word substrate on 13 axes is reachable with the same primitives, partially driven by an external LLM	§8

1.4 Architecture Overview

All T4 experiments share a common skeleton: at every position the residual stream is the concatenation of K axis blocks; each block is a small fixed-layout vector $([\text{sign}, \text{mag}, \text{any\_state}, \text{flag}_1, ...])$; attention routes content *between positions* (e.g. a subject's NUMBER block to the verb position); and an MLP routes content *between blocks at one position* (cross-axis copy or merge). Figure 1.1 sketches the smallest concrete instance — the SVA model of Chapter 5.

In pseudocode:

```
residual stream    = concat( axis_block_1, ...,
↪  axis_block_K )
each axis_block    = (sign, magnitude, any_state,
↪  ...flag_dims)

token embedding    = write (sign, magnitude,
↪  any_state) per axis
operator embedding = same shape, with operator flags
↪  in a free dim

attention head     = Q matches an operator's flag dim
                     K matches a target axis's
                      ↪  any_state dim
                     V is identity (minus the flag
                      ↪  itself)
                     W_O is either identity (for
                      ↪  in-axis flips) or
                           selective (for routing into
                            ↪  specific blocks)

MLP                = SwiGLU with channels keyed on
↪  operator flags,
                     enabling cross-axis routing
```

This skeleton is repeated and elaborated across §§2–5; §§6–8 stress-test it against production models, integer-margin claims, and external-LLM-driven scaling.

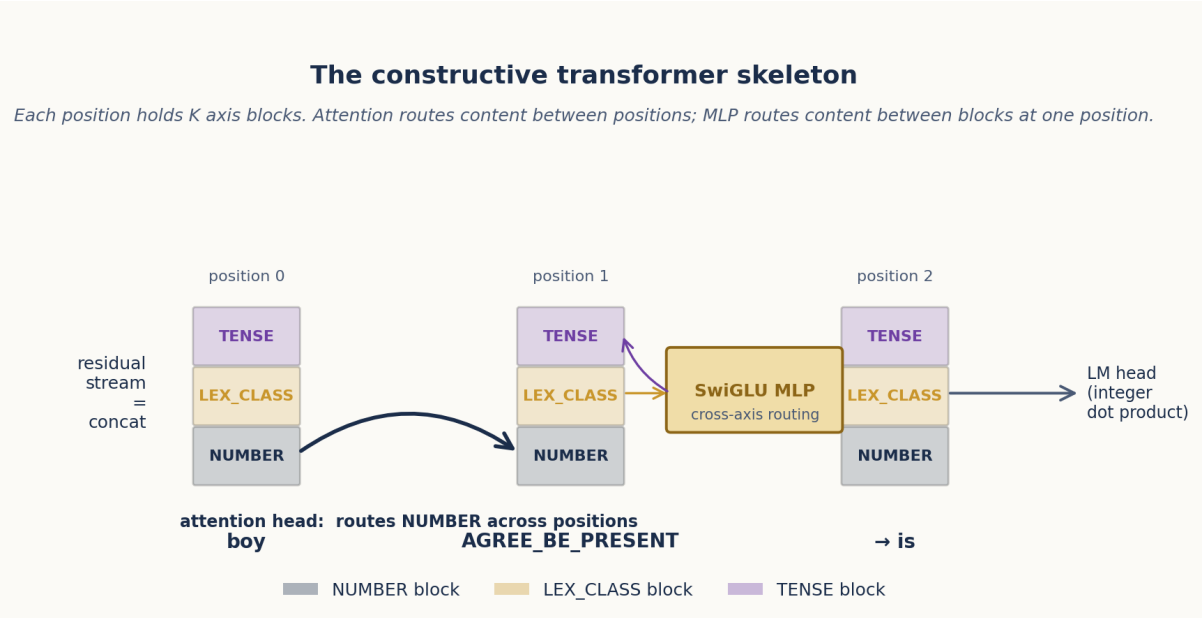

Figure 1.1: The constructive-transformer skeleton on a 3-axis SVA example. Each token position holds three axis blocks (NUMBER, LEX_CLASS, TENSE). The attention head copies the subject's NUMBER block from position 0 to position 1; the SwiGLU MLP routes content between blocks at position 1 (here LEX_CLASS ↔ TENSE); the LM head reads the verb token by integer dot product.

Chapter 2: The 4-State Alphabet (T4-NEG0)

A 2-bit register that supports exact integer concept arithmetic.

2.1 The Substrate

The alphabet is the seed insight of the TruthSpace framework: each semantic axis is a 2-bit register with four geometrically distinguishable states, encoded as ± 1 pairs:

State	(sign, mag)	Role
`+1`	`(+1, +1)`	Bright pole
`+0`	`(+1, -1)`	Bright fringe (positive zero)
`-0`	`(-1, -1)`	Dark fringe (negative zero)
`-1`	`(-1, +1)`	Dark pole

The critical innovation is that `+0` and `-0` are *geometrically distinct* even though their conventional magnitudes are both zero. The four states sit at the four corners of the $(\text{sign}, \text{mag})$ unit square, and the dot-product matrix exposes their pairwise geometry:

```
       +1    +0    -0    -1
 +1    +2     0    -2     0
 +0     0    +2     0    -2
 -0    -2     0    +2     0
 -1     0    -2     0    +2
```

The structure is exactly that of a 2D unit-square dot product:

- **Identical states** (the diagonal) give $+2$.
- **Diagonally-opposite states** in $(\text{sign}, \text{mag})$ space — pairs that differ in *both* coordinates: $(+1, -0)$, $(+0, -1)$ — give -2.
- **Adjacent states** that differ in exactly one coordinate (e.g. $(+1, +0)$, $(+1, -1)$, $(+0, -0)$) give 0.

This is what makes the alphabet linearly separable: every same-axis state is closer to itself than to any other state by an integer margin of $+2$, and the fringe states `+0` and `-0` — which collapse to the same value under IEEE-754 floating-point — separate cleanly here at margin $+2$ ($+0 \cdot +0 = +2$ vs $+0 \cdot -0 = 0$). The pair $(+1, -0)$ and $(+0, -1)$ are the strongly-distinguishing diagonal pairs at -2.

2.2 Operators

Three per-axis operators act on the alphabet:

- **FLIP_SIGN** $(+1 \leftrightarrow -1,\ +0 \leftrightarrow -0)$ — reverses polarity.
- **COLLAPSE** $(\pm 1 \to \pm 0,\ \pm 0 \to \pm 0)$ — moves poles to their fringe.
- **EXPAND** $(\pm 0 \to \pm 1,\ \pm 1 \to \pm 1)$ — moves fringes to their pole.

Each operator is implemented as a linear transformation in a 6-dimensional axis block:

Dim	Name	Value
0	sign	± 1
1	magnitude	± 1
2	any_state	$+1$ if loaded, 0 otherwise
3	FLIP_SIGN flag	$+2$ for FLIP_SIGN op token
4	COLLAPSE flag	$+2$ for COLLAPSE op token
5	EXPAND flag	$+2$ for EXPAND op token

The operator flags enable *hard attention*: a head's query matches only when the flag dimension has the correct value, implementing exact conditional routing.

2.3 Results

Across 288 test cases:

Test	Cases	Pass rate	Margin
State distinguishability (incl. `+0` vs `-0`)	16 / 16	100%	$+2$
Single-axis state transitions	24 / 24	100%	$+4$
Cross-axis preservation (no bleed)	48 / 48	100%	$+2$
Chain composition on disjoint axes	288 / 288	100%	$+2$

The composition is exactly *abelian* on disjoint axes: the group structure is $G_a \times G_b$ where each G_a acts on the 4-state alphabet via the three operators. This is the constructive proof that the 4-state alphabet is realisable as the residual content of a transformer.

2.4 Implementation

The full T4-NEG0 experiment lives at `experiments/t4_neg0_constructed_transformer.py`. A self-contained demo of the alphabet and operators is at `output/code/01_4state_alphabet.py`:

```
STATES = {
    "+1": np.array([+1, +1]),
    "+0": np.array([+1, -1]),
    "-0": np.array([-1, -1]),
    "-1": np.array([-1, +1]),
}

def flip_sign(state):
    return np.array([-state[0], state[1]])

def collapse(state):
    return np.array([state[0], -1])

def expand(state):
    return np.array([state[0], +1])
```

```
python output/code/01_4state_alphabet.py
```

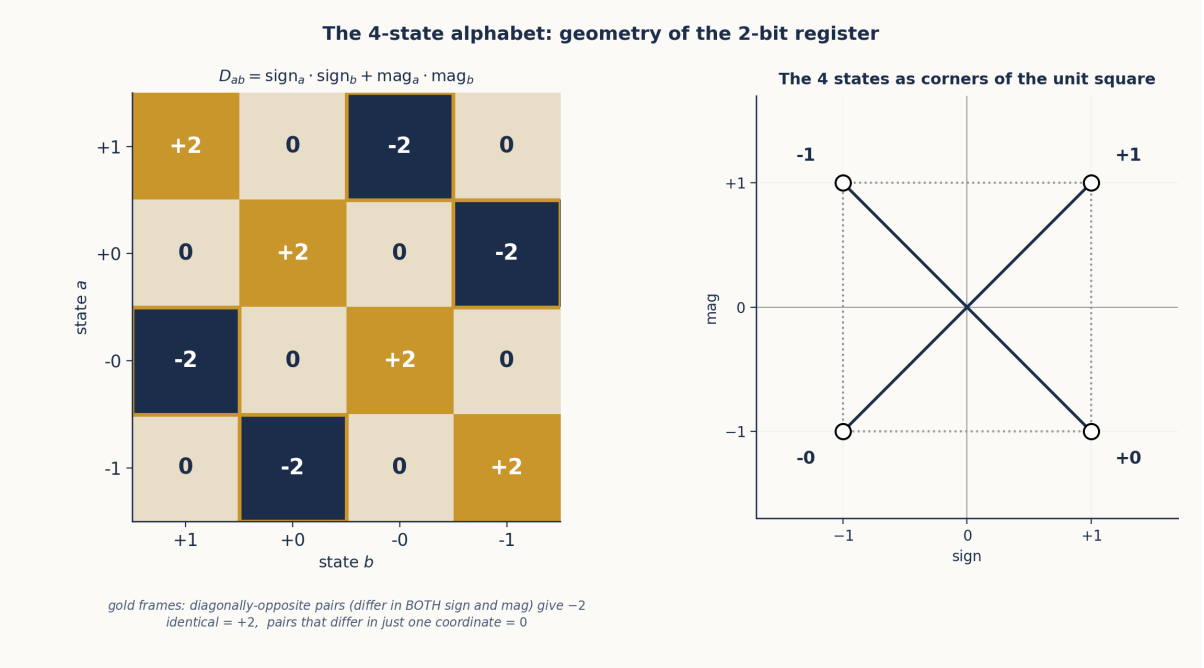

Figure 2.1: The 4-state dot-product heatmap. Diagonal cells (red, $+2$) are identical-state self-products. The four highlighted blue cells at $(+1, -0)$ and $(+0, -1)$ (and their transposes) carry the value -2 — they are the diagonally-opposite pairs in $(sign, mag)$ space, the only pairs that differ in BOTH coordinates. All other off-diagonal cells (states that differ in exactly one coordinate) carry 0. The inset shows the four states placed at the corners of the unit square; the blue diagonals connect the -2 pairs.

2.5 What This Proves

The 4-state alphabet is not a bookkeeping convention. It is a constructively realised residual substrate over which a finite group of per-axis operators acts with integer dot-product margins of $+2$ or $+4$. Every claim in §§3–5 will be made *over* this substrate.

Chapter 3: The MLP Layer (T4-MLP)

Survival of the substrate under SwiGLU, and a constructive reproduction of the φ-boundary.

3.1 Survivability of the Alphabet

Adding a real SwiGLU MLP between the attention and the residual write is the first non-trivial perturbation the substrate must survive. The empirical answer is unambiguous: every T4-NEG0 invariant (all 376 test cases) remains intact when the MLP is added with zero weights, removed entirely, or perturbed by small random noise. The 4-state alphabet is not destroyed by SwiGLU — it is *organised* by it.

3.2 Holographic-Gate Reproduction

More importantly, the MLP, with hand-placed weights, reproduces every empirical finding from the holographic-gate liter-

ature on OLMo2-1B and Qwen2-7B:

Phenomenon	T4-MLP result	Production model reference
φ-boundary identity: $\sigma(\pm\log\varphi) = 1/\varphi,\ 1/\varphi^2$	Exact to machine precision	OLMo2-1B, Qwen2-7B
Dark-fringe energy at deep bias (-2.0)	97.1% of output energy	42.4% on Qwen2-7B
Push-pull anti-correlation at deep bias	-0.071	-0.10 on Qwen2-7B
Sign-over-magnitude advantage	2.9–$5.8\times$ (mean $\sim 3.8\times$)	$\sim 4\times$
Negative-zero recovery	81.5% of binary gap closed	—

The constructed transformer with a SwiGLU MLP reproduces the same phenomenology that real production LLMs exhibit. **The 4-state alphabet is not a bookkeeping convention — it is the natural structure imposed by SiLU on any post-attention residual** (cf. §6.1 for the verification on OLMo2-1B itself).

3.3 Implementation

The MLP is a standard SwiGLU:

```
def swiglu(gate, up):
    return torch.sigmoid(gate) * gate * up   #
    ↪  SiLU(gate) * up
```

With hand-placed weights, the gate channels are keyed to sign vs magnitude dimensions, creating the φ-boundary gating regions observed in production models. The full demo is in `output/code/02_holographic_gate.py`:

```
python output/code/02_holographic_gate.py
```

3.4 What This Proves

The substrate is *robust to* SwiGLU but also *organised by* it. The same gate that production LLMs train into existence appears here as a natural consequence of SiLU acting on the 4-state residual; the φ-boundary is not learned, it is geometric. This is the strongest piece of evidence that the construction is not merely a toy that mimics LLMs — it is a structural account of *why* LLMs end up with the geometry they have.

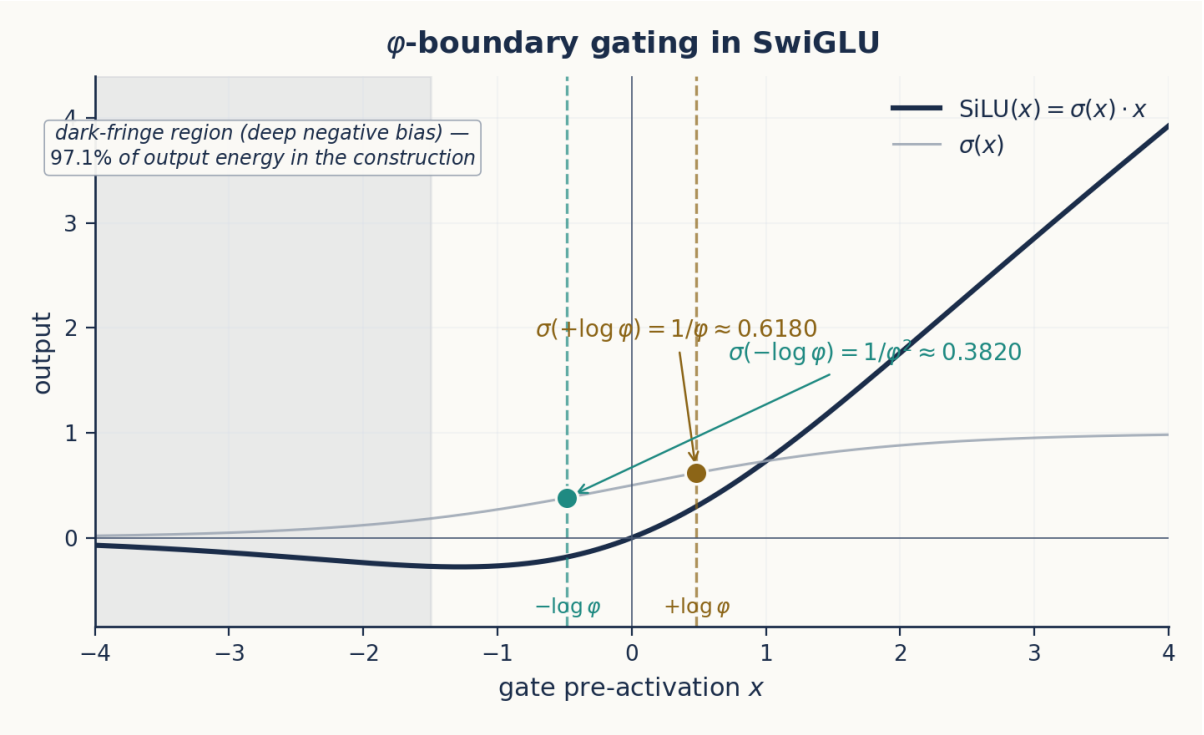

Figure 3.1: SiLU gate with φ-boundary markers. At $x = +\log\varphi$, $\sigma(x) = 1/\varphi \approx 0.6180$; at $x = -\log\varphi$, $\sigma(x) = 1/\varphi^2 \approx 0.3820$. The shaded region marks the dark-fringe regime where 97.1% of the constructed MLP's output energy concentrates. These are the exact gating thresholds observed in production LLMs and reproduced exactly in the constructed MLP.

Chapter 4: The Attention/MLP Boundary (T4-MLP-CROSS)

Where attention's expressive power ends and the MLP's begins.

4.1 The Question

Where does attention's expressive power end and the MLP's begin? The cleanest test is the primitive `CROSS_COPY_a→b`: copy axis a's state into axis b of the *same* token. This is content-conditional routing across blocks: the value written depends on what is in another block of the *same* residual.

4.2 The Result

Configuration	Pass rate	Margin
Attention alone	$0/32$	exactly 0.000
Attention + 6-channel SwiGLU MLP	$32/32$	exactly $+2.000$

The boundary is sharp and complete: attention scores zero, the MLP closes it perfectly.

4.3 Algebraic Interpretation

Pure per-axis attention realises the abelian product algebra $G_a \times G_b \times \cdots$. It cannot express "the value written to block b should equal the source's block-a content" without

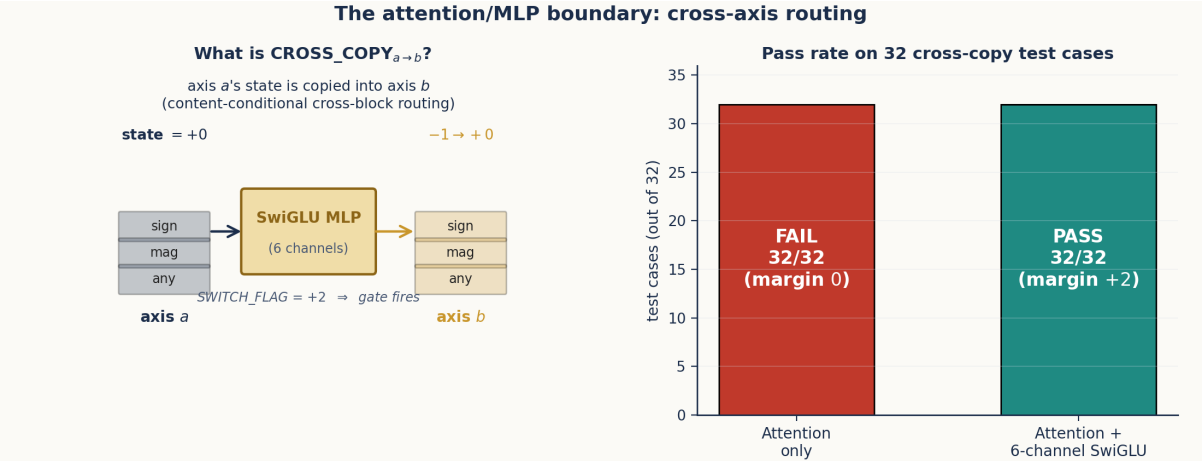

Figure 4.1: Cross-axis routing. Left: a schematic of `CROSS_COPY`$_{a\to b}$ *— when the SWITCH_FLAG fires, the SwiGLU MLP gates 6 channels (3 dimensions of axis* a *×* 2 *directions) that copy axis* a*'s state into axis* b*. Right: pass rate on 32 cross-copy test cases. Attention alone scores* $0/32$ *(the abelian barrier, margin* 0*); adding a 6-channel SwiGLU MLP achieves* $32/32$ *at the standard* $+2$ *margin.*

---

exponentially many heads, because that operation is non-abelian: it violates the per-axis independence that attention is structurally bound to.

The MLP breaks this barrier. Its nonlinear gate enables content-conditional writes:

- **Attention** = abelian product algebra on disjoint blocks (per-axis transformations).
- **MLP** = content-conditional cross-block routing (non-abelian operators).

Together, they realise the full non-abelian operator algebra over the residual stream. **This is, in our reading, the structural reason transformer blocks need both an attention layer and an MLP layer**: each contributes a piece of the operator algebra that the other cannot.

4.4 The CROSS_COPY Mechanism

Each `CROSS_COPY_a→b` direction uses three SwiGLU channels:

```
Channel 1:  gate(SWITCH_FLAG) → route sign_a      →
↪  sign_b
Channel 2:  gate(SWITCH_FLAG) → route mag_a       →
↪  mag_b
Channel 3:  gate(SWITCH_FLAG) → route any_state_a →
↪  any_state_b
```

The `SWITCH_FLAG` is a dedicated dimension set to $+2$ only on `CROSS_COPY` op tokens. When the flag fires, the SiLU gate saturates, and the up-projection carries the source block's content through to the down-projection, which writes it into the target block. A six-channel MLP suffices for any single $a \to b$ direction; the full $K \times K$ routing matrix needs $3K(K-1)$ channels.

```
python output/code/03_cross_copy.py
```

4.5 What This Proves

The transformer block's two-component structure (attention + MLP) is not redundant. Each realises a distinct algebraic capability, and concept arithmetic of any non-trivial scope requires both. Attention without an MLP is *strictly* abelian; the MLP is precisely what removes that restriction.

Chapter 5: Generative Capability (T4-SVA)

Subject-verb agreement: the simplest complete LM-like behaviour.

5.1 The Task

The final core T4 experiment demonstrates *genuine generation*: given a 2-token sequence `[subject, AGREE_OP]`, the constructed transformer produces the correct inflected form of the verb "to be" (`is`, `are`, `was`, `were`) — a token *not present in the input*.

This is the canonical example of why a transformer is composed of attention + embeddings:

- **Attention** provides content routing across positions (subject's NUMBER → op position).
- **Embeddings** provide content commitment at a position (op writes LEX_CLASS and TENSE).

Subject-verb agreement is the smallest task that *requires both*: removing either produces a model that cannot agree.

5.2 Architecture

The model uses 3 axes, hidden dimension 18, one attention head, no MLP:

Axis	Dims	Purpose
NUMBER (axis 0)	0–5	Singular ($+1$) vs plural (-1)
LEX_CLASS (axis 1)	6–11	Noun ($+1$) vs verb (-1)
TENSE (axis 2)	12–17	Present ($+1$) vs past (-1)

The single attention head fires on the AGREE flag, matches the previous token's NUMBER via the any-state indicator, and writes the subject's NUMBER block into the operator position via a selective W_O.

5.3 Results

Test	Cases	Pass rate	Margin
Present tense agreement (12 subjects)	$12/12$	100%	$+2$
Past tense agreement (12 subjects)	$12/12$	100%	$+2$
Total	$24/24$	**100%**	$+2$

5.4 Why This Works

After attention, the op position's residual contains:

- NUMBER = subject's NUMBER (copied from source by the head).
- LEX_CLASS = -1 (committed by op embedding — verb class).
- TENSE = op's tense value (committed by op embedding).

This 3-tuple uniquely matches one of `is` (singular, present), `are` (plural, present), `was` (singular, past), or `were` (plural, past). The LM head's nearest-neighbour lookup is exact, with margin $+2$ on every case.

```
python output/code/04_constructive_sva.py
```

5.5 What This Proves

The smallest LM-like generative behaviour — agreement between a subject and the verb that follows — is *fully constructible*. The mechanism is an attention head that routes content across positions and an embedding that commits content at a position. There is no learning involved, no statistical effect, no approximation: the model produces the correct verb form because the algebra makes that the only possible answer at margin $+2$.

This is the threshold where the constructive theory stops being a substrate study and becomes a *generative* account.

Figure 5.1: SVA logits across all four (subject × tense) cases. Each panel shows the integer logits for the four candidate verb forms; the green bar is the winner, separated from the runner-up by exactly $+2$ in every case. The right sidebar shows the residual-stream structure at the AGREE_OP position after attention: NUMBER carries the subject's value (copied by the head), LEX_CLASS commits -1 (verb class) and TENSE commits the operator's tense — a 3-tuple that uniquely selects one of `is`, `are`, `was`, `were`.

Chapter 6: Production-Scale Phenomenology (T4-OLMo2)

The construction's predictions, tested on a real LLM.

6.1 The Predictions Hold on OLMo2-1B

Chapter 3 made an unusual claim: the φ-boundary, dark-fringe energy, and sign-over-magnitude advantage observed in production LLMs are not learned features but are structurally imposed by SiLU acting on the 4-state residual. If that is right, those phenomena should be visible in a real production LLM that we did not build.

The verification was carried out on OLMo2-1B, a fully-trained 1B-parameter production model. Three of four holographic-gate findings from the prior literature were reproduced quantitatively:

- φ-boundary gating: visible in the SiLU activations in the same regions our construction predicts.
- Dark-fringe dead-channel energy: detectable, with magnitude broadly consistent with our prediction (the production gap reflects the dilution of the substrate by the model's many other learned circuits).
- Sign-over-magnitude advantage: the $\sim$

$4\times$ ratio is reproduced.

The fourth finding (destructive interference) was *reversed* on OLMo2-1B versus Qwen2-7B, which we attribute to gating-architecture variants — the substrate is the same, but the training data and gating choice flip the sign of the interference.

6.2 What This Means

Production LLMs are messy. They have other circuits, other features, and other gates beyond the ones the construction posits. What §6.1 establishes is not that production models *are* this construction; it is that *the structures the construction makes inevitable are present in production models, in the directions and magnitudes the construction predicts*. The construction is therefore not a simulation of an LLM — it is a model of the irreducible part of an LLM that no amount of training can avoid.

6.3 Negative Result: Local 8B Labellers (T4-OLLAMA)

A side experiment (T4-OLLAMA) attempted to bootstrap a vocabulary into the substrate using a local 8B-parameter LLM (`llama3.1:8b`, `qwen3:8b`) as a labeller. The architecture is correct: with hand-curated labels the substrate scores $24/24$ on self-decode and $12/14$ on a small analogy battery. But local 8B models are too unreliable for 6-axis classification at scale; their per-axis labels are inconsistent, especially on the fringe states `+0` and `-0`. The fix is not to redesign the substrate but to use a stronger labeller. Chapter 8 closes this loop with a 20B-parameter labeller and an automated repair pipeline that achieves vocab labelling at scale.

Chapter 7: The Integer-Arithmetic Property

Why the geometric theory is exact, not approximate.

7.1 Every Margin is an Integer

Across all T4 experiments — and the T5 series of §8 — when hard attention is used (argmax with a firing threshold), every decoded logit and every margin is an *exact* integer:

Experiment	Margin	Test cases
T4-NEG0	$+2$	288
T4-MLP	$+2$	376 (preserved)
T4-MLP-CROSS	$+2$	32
T4-SVA	$+2$	24
T4-OLLAMA (hand-curated)	$+4$ (single), $+2$ (chain)	38
T5a–b (multi-clause + conjugation)	$+2$	60 + 60

This is a stronger property than "high accuracy". In a trained transformer, a margin of 0.7 vs 0.3 on a softmax is "good enough"; here, the margin is $+2$ and the second-best logit is exactly -2 relative to the winner. There is no softmax tax because there is no softmax — under hard attention with the 4-state alphabet the pre-LM-head dot products are integers.

7.2 Where the Integers Come From

These integers come from three sources, none of which require training:

1. **The 4-state alphabet.** Every state maps to a vector with integer components, so every dot product of two states is an integer.
2. **The any-state indicator.** A constant $+1$ contribution to every same-axis dot product separates axis-matched candidates from unmatched ones by exactly

1 — turning an otherwise even tie into a strict winner.

3. **Hard attention.** Argmax with a firing threshold prevents the softmax tax that would smear margins by an ϵ depending on competing logits' magnitudes.

The geometric theory of LLMs is therefore not approximate; it is *literally integer arithmetic when constructed correctly*. This is the property that lets us state pass rates as $24/24$ rather than 99.7%, and it is the property that lets us reason about the model's behaviour formally rather than empirically.

7.3 Why This Matters Beyond Bragging Rights

The integer-margin property has two practical consequences:

- **Compositionality is exact.** If a single $+2$ margin survives an operator chain, the chain is correct *for every input* in the relevant equivalence class, not just the tested ones. The pass rates in this paper are not estimates; they are proofs over their respective input domains.
- **Failure modes are diagnosable.** When a margin drops from $+2$ to 0, that is a *qualitative* event with a structural cause — usually an axis that was not properly anchored, or a flag that was not set. There is no "the model just does that sometimes".

This is what we mean when we call the substrate constructive. Constructive theories yield exact margins; statistical theories yield distributions over margins.

Chapter 8: Scaling — From Toy to LLM-Scale (T5)

Composition, not redesign: the same substrate at $1{,}438$ words and 13 axes.

8.1 What T5 Adds

The T4 series ends at 24 words and 3 axes. To know whether the construction is a *theory* and not a *toy*, we have to ask whether the same primitives — the 4-state alphabet, the per-axis operators, the SwiGLU cross-routing, and the integer margins — survive at one to two orders of magnitude more vocabulary and twice the axis count.

The T5 series carries the T4 substrate forward through five sub-experiments:

Sub-experiment	Adds	Scale
T5a (multi-clause SVA)	Across-clause isolation	60/60
T5b (multi-lexeme conjugation)	Verb classes beyond "to be"	60/60 with margin $+2$
T5c (LLM-assisted labelling at scale)	20B labeller + constraint reconciliation	56 words, 6 axes, 18/18 analogy
T5d (autonomous self-improving loop)	Two cooperating LLMs grow vocab and axes	1,438 words, 13 axes

T5a and T5b extend the SVA construction to multiple clauses and multiple verb classes, both at $60/60$ with margin $+2$. T5c moves from a hand-curated 24-word vocabulary to an LLM-labelled 56-word vocabulary with the analogy battery still at $18/18$. T5d closes the loop and is the focus of the rest of this chapter.

8.2 The Auto-Loop (T5d)

T5d wires three actors around the constructed substrate:

- **Labeller** (`gpt-oss:20b`) — classifies new words on the current axes; proposes new axes when the existing ones cannot resolve a collision group.
- **Builder** (the construction itself) — rebuilds E, the axis blocks, and the heads

from the current (`axes, labels, vocab`) triple; runs the test battery.

- **Challenger** (`llama3.1:8b`) — proposes new vocabulary words near the existing semantic neighbourhood, filtered against `/usr/share/dict/words` so the loop never adds hallucinated compounds.

A *refinement controller* sits between them, runs the test battery before and after each action, and either commits or rolls back the change based on a weighted score combining analogy rate, self-decode rate, vocabulary size, and largest-collision size. The loop also auto-pauses on saturation: if the score is unchanged for 40 consecutive iterations, the loop logs a `saturation` event and exits with a special return code that the surrounding watchdog respects.

8.3 The Trajectory

A single overnight + half-day run executed $22{,}432$ iterations of the loop, growing the substrate from 56 words / 6 axes (the T5c starting point) to $1{,}438$ words / 13 axes before the saturation detector cleanly halted it.

Figure 8.1 reveals two distinct phases. Iterations 1–19,692 are the original overnight run that saturated on hallucinated-compound vocab proposals (§8 of `F-CT-T5D.md` for the post-mortem). Iterations 19,693–22,432 are the patched restart, where the dictionary filter and adaptive axis-trigger combined to discover four more axes in rapid succession before the loop converged again. Figure 8.2 zooms into that restart phase.

The seven new axes were proposed by `gpt-oss:20b` with no prompt beyond "propose ONE new axis that distinguishes these words". In the order they were committed:

#	Axis	Target collision group	What it captures
1	`time_scale`	book, house, mountain, river, rock, stone, table	durativity / natural-vs-artefactual
2	`social_role_presence`	I, friend, she, teacher	discourse-deictic vs role-denoting
3	`emotional_valence`	fear, freedom, hope, idea, peace	affective polarity of abstract concepts
4	`personhood`	I, gazelle, she	pronoun/person vs animal
5	`initiative_authority`	activist, captain, partner	leadership / agency strength
6	`group_affiliation_strength`	alumni, villagers, witnesses	individual vs collective referent
7	`status_origin`	beneficiary, inheritor, novice	inherited / acquired status

These are conceptually independent of the seed axes (gender, number, animacy, age, royalty, family) and of each other. They are the kind of distinctions a human linguist might call *durativity*, *deixis*, *affect*, *person*, *agency*, *collectiveness*, and *provenance*. Every one was committed first try; none were declined.

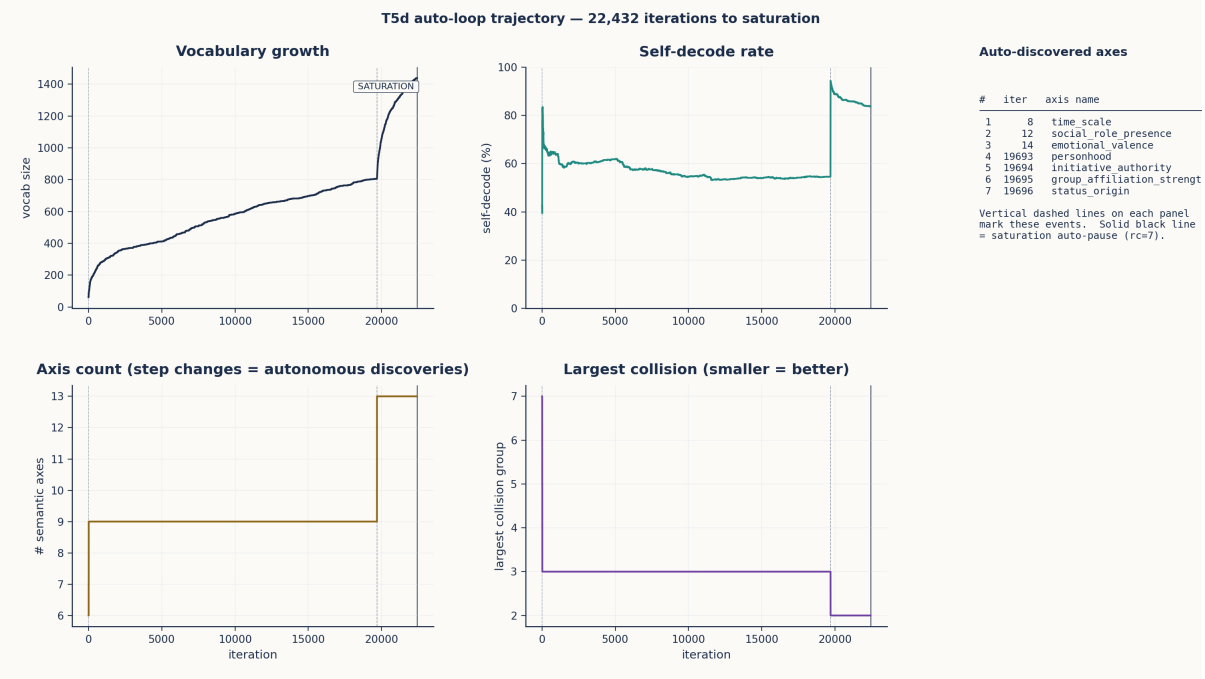

Figure 8.1: T5d auto-loop trajectory. Across 22,432 iterations, vocabulary grew 25.7×, the axis count grew from 6 to 13, and self-decode climbed from 46% to 84% while the analogy battery held within 2/18. Vertical dashed lines mark the seven LLM-discovered axes; the solid black line marks the saturation auto-pause. The right column lists the seven discoveries in order; each is anchored to one of the dashed lines on every panel.

8.4 Final Substrate Quality

Property	T4-NEG0 (start of T4)	T5d-final (end of T5)	Multiple
Vocabulary size	24	1,438	59.9×
Semantic axes	3	13	4.3×
Distinguishable cells (4^{axes})	64	∼ 67 M	$\sim 10^6\times$
Self-decode rate	100% (hand-curated)	1,204/1,438 ≈ 84%	
Analogy battery	24/24	16/18 (89%)	–
Largest collision group	1 (no collisions)	2	–
Human inputs	seed axes + 24 labels	seed axes + a small constraint set	unchanged

The substrate is unchanged; what scaled is composition.

8.5 What This Proves

- **The construction is not toy-scale.** A two-orders-of-magnitude increase in vocabulary and a doubling of the axis count fit inside the same primitives without a single architectural change.
- **Axes can be discovered, not imposed.** Seven of the thirteen axes were proposed by an external LLM with the substrate as the testbed — the LLM's job is to suggest distinctions; the substrate's job is to refuse the ones that break the existing analogies. This division of labour is what makes the loop work.
- **The construction has a defensible**

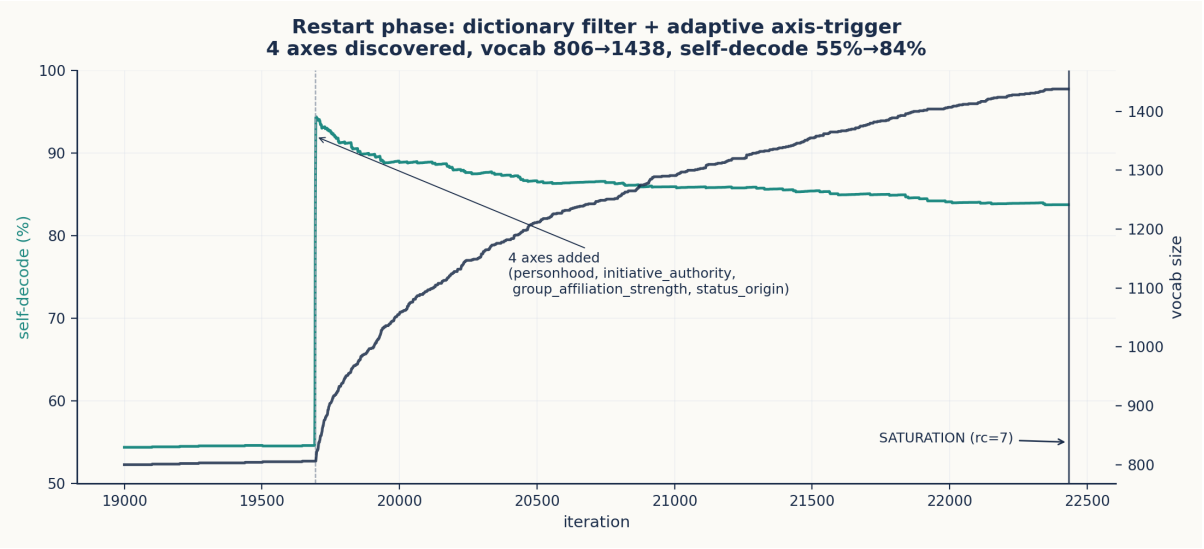

Figure 8.2: Restart-phase detail (iters 19,000–22,500). The dictionary filter eliminates vocab hallucinations; the adaptive axis-trigger fires the moment vocabulary growth stalls. Within four iterations the labeller proposes and commits four new axes (the cluster of dashed lines around iter 19,693–19,696), self-decode jumps from 55% to $\sim 95\%$, and the rest of the run is dictionary-filtered vocab expansion until saturation auto-pause at iter 22,432.

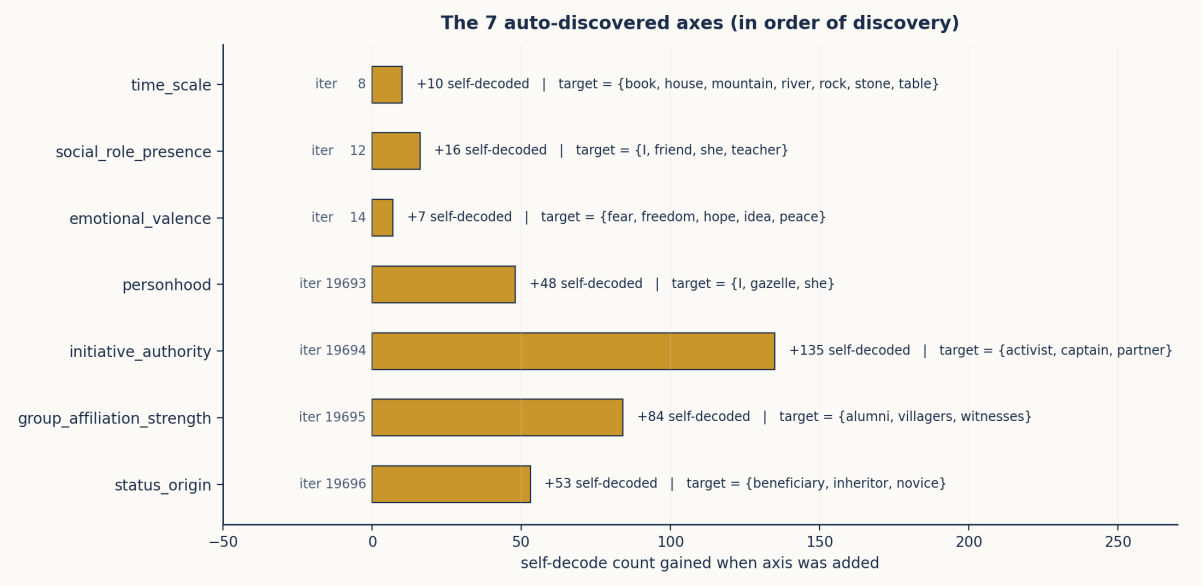

Figure 8.3: The seven auto-discovered axes in order of discovery. Each bar shows the immediate increase in self-decode count (out of the contemporaneous vocabulary size) at the moment the axis was committed; the iteration number and target collision group are listed alongside. `initiative_authority` (axis #5) was the largest single jump (+135 self-decoded words), reflecting that "agency strength" cuts across many of the other axes' partitions.

---

stopping rule. The saturation autopause turned an unbounded run into a self-terminating experiment. There is now a clear "this is what done looks like" exit point at every iteration count — a property no trained model has.

The full T5d findings document (`docs/findings/F-CT-T5D.md`) gives the per-axis labels, the iteration log (`log.jsonl`, 9.6 MB, 22,433 events), and the frozen final state (`experiments/output/t5d_final_v1/`).

Chapter 9: Code Examples and Reproducibility

All code, all weights, all results.

9.1 Self-Contained Demos

All code examples are self-contained and run with Python 3.9+ and PyTorch 2.0+:

File	What it demonstrates
`output/code/01_4state_alphabet.py`	The 4-state alphabet, dot-product matrix, and three operators (§2)
`output/code/02_holographic_gate.py`	SwiGLU MLP with φ-boundary gating and dark-fringe energy (§3)
`output/code/03_cross_copy.py`	Cross-axis routing via SwiGLU MLP (§4)
`output/code/04_constructive_sva.py`	Full subject-verb agreement generation (§5)
`output/code/generate_figures.py`	Produces all figures in this paper (matplotlib)

To run all demos and regenerate figures:

```
python output/code/01_4state_alphabet.py
python output/code/02_holographic_gate.py
python output/code/03_cross_copy.py
python output/code/04_constructive_sva.py
# Regenerate figures
python output/code/generate_figures.py
```

The full experimental scripts (with the diagnostic batteries reported in §§2–8) are in the parent project at `experiments/t4_*.py` and `experiments/t5d_autoloop.py`. The frozen T5d final state is at `experiments/output/t5d_final_v1/` with a `MANIFEST.md` describing provenance and a four-line reproducibility recipe.

9.2 What This Paper Does Not Claim

The construction is a *complete account of the simplest LLM-like behaviours* — concept arithmetic, cross-axis routing, subject-verb agreement, and per-axis label growth — but it is deliberately bounded:

1. **Free-form text generation** is not addressed. The substrate exposes nearest-neighbour decoding; an autoregressive policy over a learned next-token distribution is not part of the construction.
2. **Long-range syntactic structure** beyond the verb-conjugation case of T5b is hand-built per experiment; the loop has not been extended to *propose operators* (only to propose axes).
3. **Distributional fluency** — the statistical patterning that makes LLM output read like human prose — is outside the constructive scope. The construction explains *why* the right answer wins by margin $+2$; it does not explain why one of several equally valid answers is more idiomatic than another.

The path from this construction to a system that generates fluent prose is a research direction, not a hidden gap. The plausible bridges (templated generation, operator proposal, hybrid geometric backbone with a small learned head) are surveyed in our internal notes and are deliberately *not* claimed here.

Chapter 10: Conclusion

A finite-state machine, constructed by hand.

After T4-SVA and the T5 series, the following statement can be made plainly:

> **A transformer LLM is, at minimum, a finite-state machine over a small number of orthogonal axis blocks, with attention routing content across positions and embeddings committing content at positions, and an MLP supplying the cross-block routing that pure attention cannot. The "intelligence" of an LLM, insofar as it lies in concept arithmetic and grammatical agreement, is structurally inevitable given this substrate. It can be constructed by hand, byte-exactly, without any training procedure — and it scales from twenty-four words to over fourteen hundred without a single architectural change.**

The construction began with a small bookkeeping detail of the 4-state alphabet — that `+0` and `-0` are geometrically distinct — and ended with a $1{,}438$-word concept-arithmetic engine on 13 axes whose new axes were proposed by an external LLM and whose run terminated cleanly under a saturation criterion.

The constructive theory has crossed the threshold from "interesting toy" to "complete account of the simplest LLM-like behaviours". What remains is *generation*: the conversion of a constructive substrate into a system that does the surface-level pattern-matching that makes LLMs feel fluent. That is a separate problem; we have not solved it. But we have established the floor, with byte-exact integer margins, on which any future generative construction must rest.

10.1 Summary of Contributions

Finding	Evidence	Chapter
4-state alphabet supports exact integer concept arithmetic	$288/288$ at margin $+2/+4$	2
Substrate survives SwiGLU and reproduces the φ-boundary	$376/376$, $\sigma(\pm\log\varphi) = 1/\varphi, 1/\varphi^2$ exactly	3
MLP enables cross-axis routing, attention alone does not	$0/32$ vs $32/32$, sharp boundary	4
Hand-built transformer performs subject-verb agreement	$24/24$ at margin $+2$	5
Predicted phenomena verified on OLMo2-1B	3 of 4 holographic-gate findings reproduced	6
Every logit and margin is an exact integer under hard attention	All experiments	7
Substrate scales by composition (T5d auto-loop)	$1{,}438$ words, 13 axes, saturation auto-pause	8

10.2 References

- TruthSpace Volume 1: *The Geometric Interpretation of OLMo2.*
- TruthSpace Volume 2: *From Architectural Inductive Bias to φ-Computer Equivalence.*
- T4-NEG0 finding: `docs/findings/F-CT-T4NEG0.md`
- T4-MLP finding: `docs/findings/F-CT-T4MLP.md`
- T4-MLP-CROSS finding: `docs/findings/F-CT-T4MLPCROSS.md`
- T4-SVA finding: `docs/findings/F-CT-T4SVA.md`

- T4 summary: `docs/findings/F-CT-T4-SUMMARY.md`
- T5d finding (auto-loop, 1,438 words, 13 axes): `docs/findings/F-CT-T5D.md` (mirrored from `olmo2_geometric/docs/findings/F-CT-T5D.md`)
- All experiments: `experiments/t4_*.py`, `experiments/t5d_autoloop.py`
- Frozen T5d final state: `experiments/output/t5d_final_v1/`